

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl

Informatik, Universität Trier

Willkommen in der ersten Reihe

- Heute teilweise ziemlich kleine Fonts in den Folien



Heute

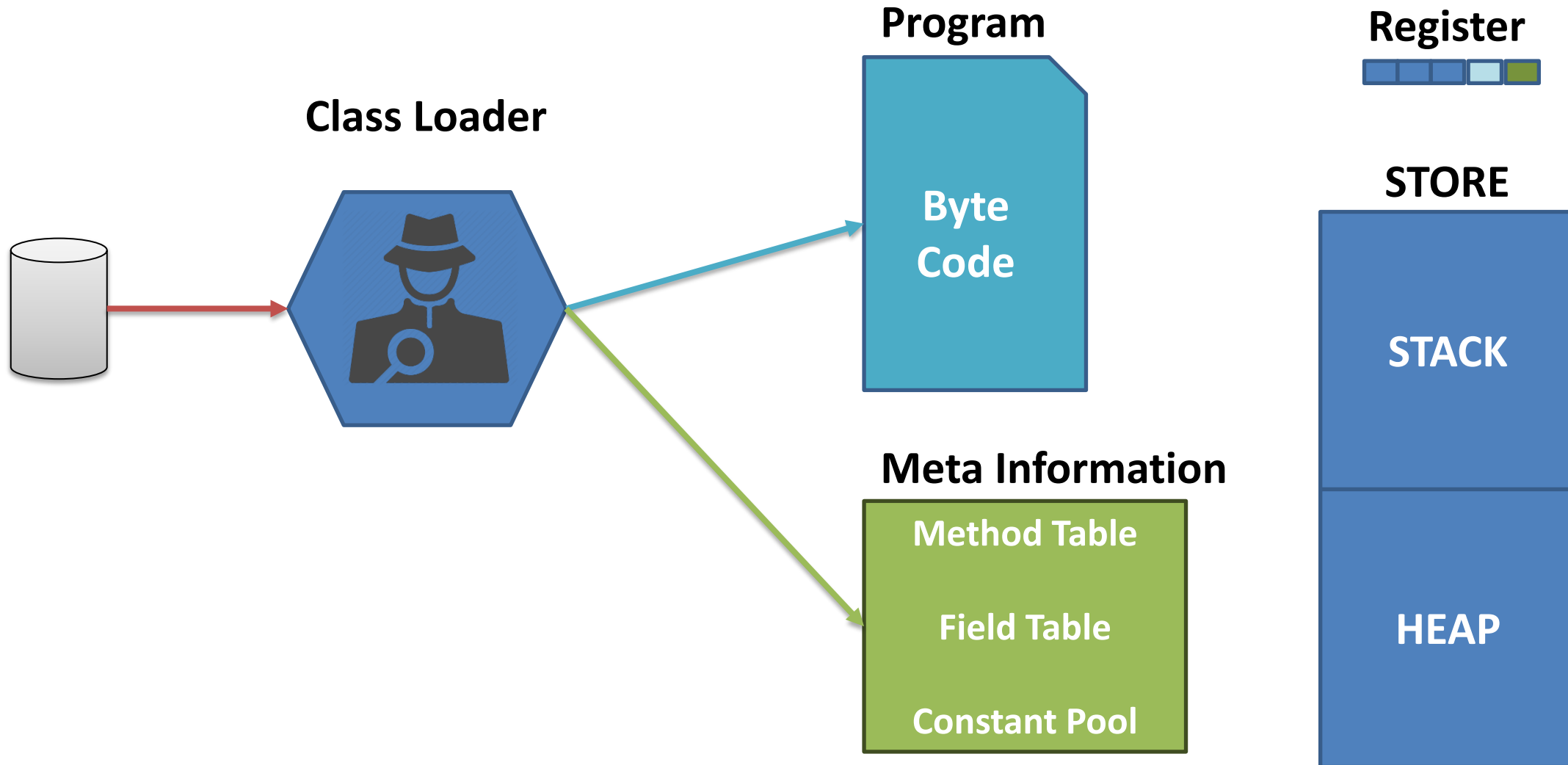
- Die Java Virtual Machine
- Übersetzung von Java

Quelle: *A Formal Introduction to the Compilation of Java*, Stephan Diehl, in "Software - Practice and Experience", Vol. 28(3), 1998. [Diehl98]



Java Virtual Machine (JVM)

Komponenten



Java Virtual Machine (JVM)

Komponenten

- Class File Loader γ
 - liefert Bytecode und Meta-Informationen
- Datenspeicher
 - STORE aufgeteilt in STACK und HEAP

$STORE : N \rightarrow Byte \times Byte \times Byte \times Byte$

$STACK : N \rightarrow Byte \times Byte \times Byte \times Byte$

$HEAP : N \rightarrow Byte \times Byte \times Byte \times Byte$

$dom(STACK) \cap dom(HEAP) = \emptyset$

- Execution Environment
 - Register: $PC, VARS, OPTOP, FRAME, CC$

Bei TRAM: PC, PP, TOP, FP



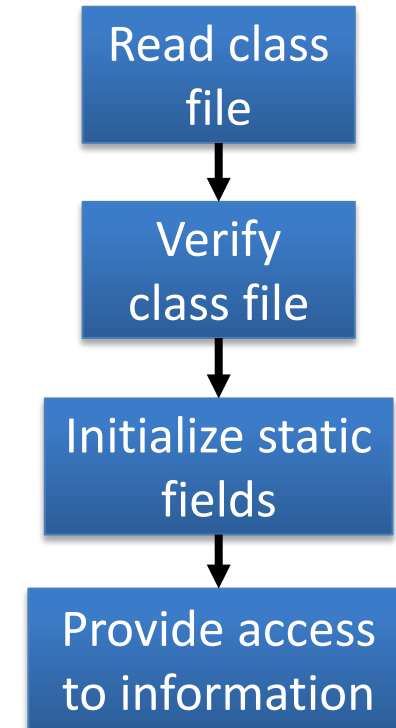
Aktuelle Klasse



Class File Loader

Der Class File Loader führt für jede (verwendete) Klasse folgende Schritte aus:

- liest die Klassendatei (Bytecode + Meta-Informationen)
- überprüft Sicherheit der Klasse (Bytecode Verification), z.B. Initialisierung, Sprungadressen
- initialisiert statische Felder der Klasse
- **stellt dem Rest der JVM Meta-Informationen über Konstanten, Felder, Methoden und Attribute der Klasse bereit.**



Classfile /C:/Users/Diehl/Desktop/Vorlesungen/Programmierung/Programmierung II/Beispiele/Zeit.class

Last modified 28.05.2019; size 312 bytes

MD5 checksum 449ea687e4f818cb455e5b0dcecf451e

Compiled from "Zeit.java"

public class Zeit implements java.io.Serializable

minor version: 0

major version: 55

flags: (0x0021) ACC_PUBLIC, ACC_SUPER

this_class: #4 // Zeit

super_class: #5 // java/lang/Object

interfaces: 1, fields: 3, methods: 1, attributes: 1

Constant pool:

#1 = Methodref #5.#17 // java/lang/Object."<init>":()V

#2 = Fieldref #4.#18 // Zeit.stunde:I

#3 = Fieldref #4.#19 // Zeit.minute:I

#4 = Class #20 // Zeit

#5 = Class #21 // java/lang/Object

#6 = Class #22 // java/io/Serializable

#7 = Utf8 stunde

#8 = Utf8 I

#9 = Utf8 minute

#10 = Utf8 tag

#11 = Utf8 <init>

#12 = Utf8 (II)V

#13 = Utf8 Code

#14 = Utf8 LineNumberTable

#15 = Utf8 SourceFile

#16 = Utf8 Zeit.java

#17 = NameAndType #11:#23 // "<init>":()V

#18 = NameAndType #7:#8 // stunde:I

#19 = NameAndType #9:#8 // minute:I

#20 = Utf8 Zeit

#21 = Utf8 java/lang/Object

#22 = Utf8 java/io/Serializable

#23 = Utf8 ()V

Ausgabe von javap

```
{
  int stunde;
    descriptor: I
    flags: (0x0000)

  int minute;
    descriptor: I
    flags: (0x0000)

  int tag;
    descriptor: I
    flags: (0x0000)

  Zeit(int, int);
    descriptor: (II)V
    flags: (0x0000)
    Code:
      stack=2, locals=3, args_size=3
      0: aload_0
      1: invokespecial #1 // Method
java/lang/Object."<init>":()V
      4: aload_0
      5: iload_1
      6: putfield #2 // Field stunde:I
      9: aload_0
     10: iload_2
     11: putfield #3 // Field minute:I
     14: return
    LineNumberTable:
      line 12: 0
      line 13: 4
      line 14: 9
      line 15: 14
}
```

SourceFile: "Zeit.java"

Class File Loader

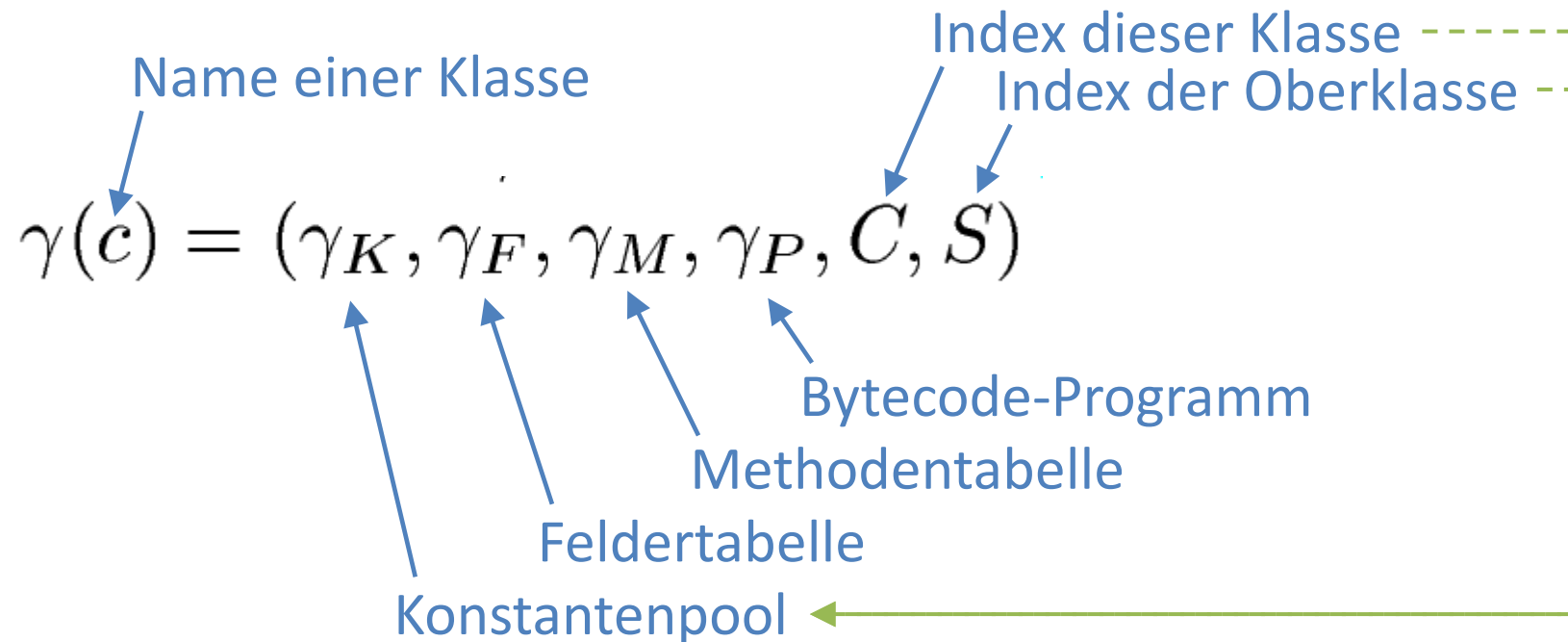
$$\Gamma_K = N \rightarrow \text{Number} \cup \text{String} \cup (\text{Number} \times \text{String}) \cup (\text{String} \times \text{String})$$

$$\Gamma_F = \text{String} \rightarrow N \times \{\text{static}, \text{dynamic}\} \times N$$

$$\Gamma_M = \text{String} \rightarrow \text{String} \times N \times N \times N \times \{\text{static}, \text{dynamic}\} \times N$$

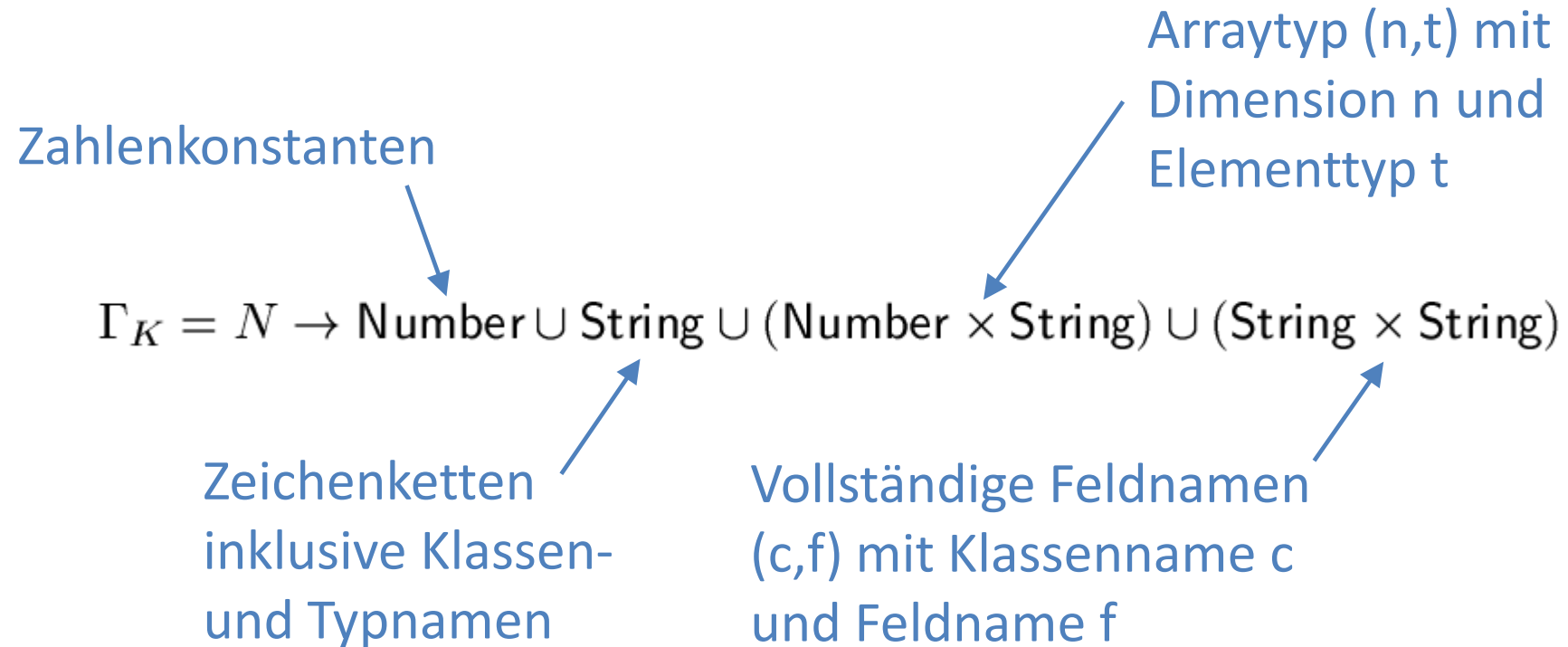
$$\Gamma_P = N \rightarrow \text{Byte}$$

Class Loader: $\gamma : \text{String} \rightarrow \Gamma_K \times \Gamma_F \times \Gamma_M \times \Gamma_P \times N \times N$



Class File Loader

Konstantenpool



Class File Loader

Feldertabelle

$$\Gamma_F = \text{String} \rightarrow N \times \{static, dynamic\} \times N$$

Name einer Variablen

$$\gamma_F(f) = (t, sd, a)$$

Index des Feldtyps
im Konstantenpool

Klassen- oder Objektvariable

Adresse, falls
Klassenvariable (static)
oder Offset, falls
Objektvariable (dynamic)

Class File Loader

Methodentabelle

$$\Gamma_M = \text{String} \rightarrow \text{String} \times N \times N \times N \times \{static, dynamic\} \times N$$

Name einer Methode

Adresse des Bytecodes dieser Methode

Klassen- oder Objektmethode

$$\gamma_M(m) = (c, p, n, l, sd, ms)$$

Klassenname

Anzahl maximal
benötigter Kellerzellen

Anzahl der Argumente
der Methode

Anzahl echter, lokaler Variablen
(exklusive **this** und Argumente)

Class File Loader

Bytecode-Programm

- Sequenz von Zellen der Größe eines Bytes

$\gamma_P(p)$ = Byte an Adresse p

Instruktionen und deren Argumente sind in
aufeinander folgenden Zellen gespeichert

Class File Loader

Execution Loop der JVM

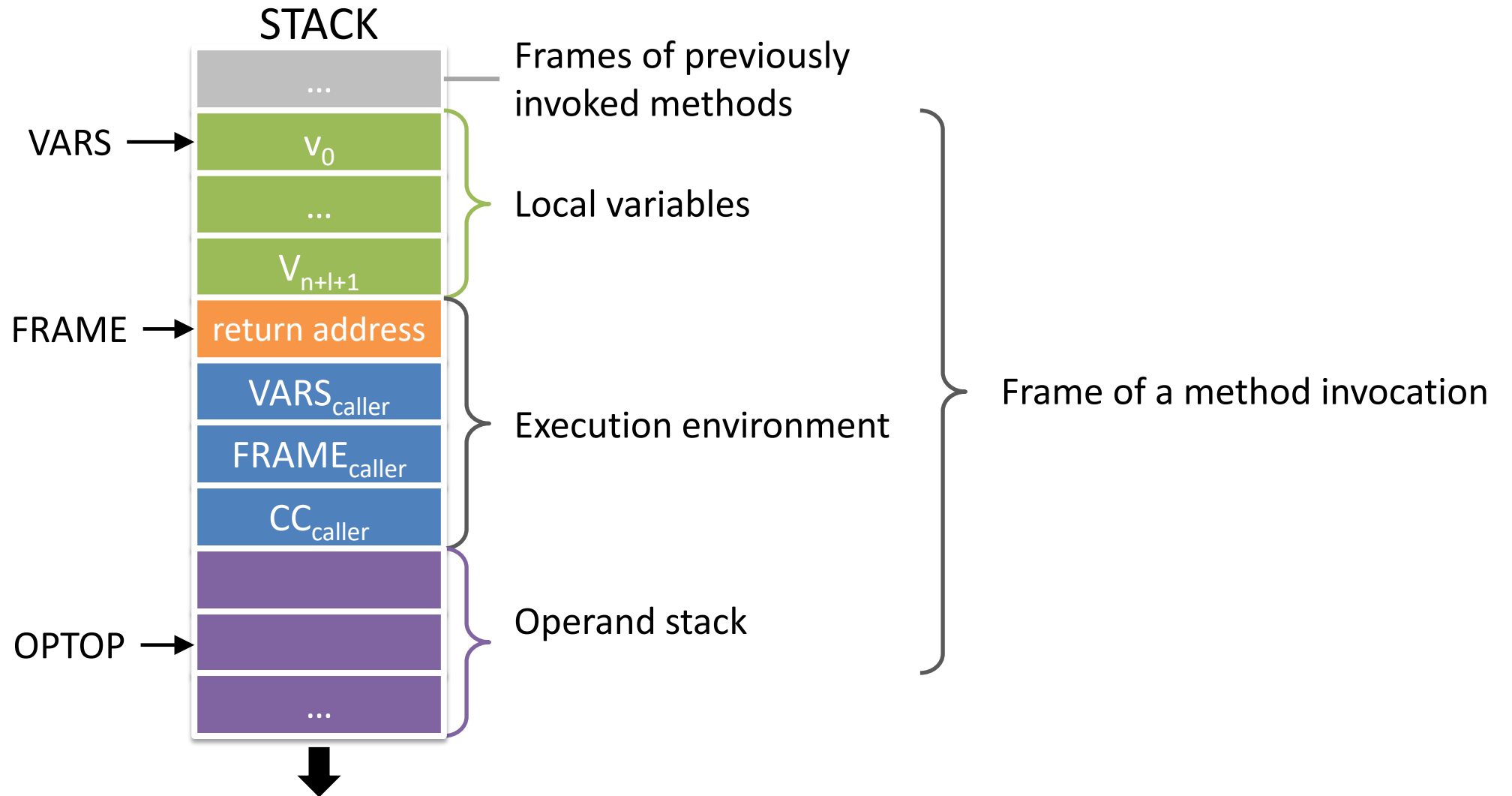
```
do forever {  
    execute ( $\gamma_P^{CC}(PC)$ ) ;  
}
```

Register: Current Class

Register: Program Counter

Instruktionen verändern den Program Counter
entsprechend ihrer Funktionalität

Keller und Kellerregister



TassKaf

- Teilmenge von Java
- Wichtige Aspekte abgedeckt, z.B.:
 - Integer Arithmetik und einfache Ausdrücke
 - Klassen und Vererbung
 - Arrays
 - Überladen von Methoden
- Nicht abgedeckt, z.B.:
 - Casts
 - Interfaces
 - Threads
 - Exceptions



Syntax sowie diverse
Beispiele in [Diehl98]

Compiler

TassKaf nach Java Bytecode

- Kodeerzeugung



- Beispiel (Integer-Addition):

– $code\ (e_1 + e_2)\ \rho = \underbrace{code\ e_1\ \rho;\ code\ e_2\ \rho;\ iadd}_{\text{Java Bytecode}}$

Instruktionen der JVM

- Instruktionen ohne Argumente
 - `iadd`, `iconst_0`, ...
- Instruktionen mit Argumenten
 - `istore v`, `goto a`, `if_icmpeq a1 a2`, ...
- Varianten von Instruktionen für verschiedene Typen
 - `istore v`, `astore v`, ...
- In einer Instruktion kodierte Argumente
 - `iconst_n` mit $n = 0, \dots, 5$
- <http://docs.oracle.com/javase/specs/>
 - Java Virtual Machine Specification

Argumente befinden sich in den nachfolgenden Zellen

n abhängig von der jeweiligen Instruktion

Instruktionen der JVM

Ausdrücke

Integer Arithmetics

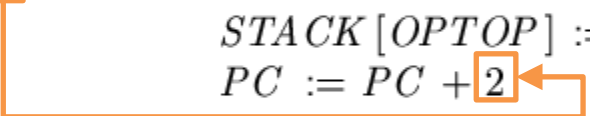
Instruction	Meaning	Before	After
iadd	$STACK[OPTOP - 1] :=$ $STACK[OPTOP - 1] + STACK[OPTOP];$ $OPTOP := OPTOP - 1;$ $PC := PC + 1$	(N,N)	(N)
analogous: isub , imul , idiv , iand , ior		Verarbeitet die beiden oberen Zahlen ausgehend von OPTOP und schreibt das Ergebnis als obersten Wert auf den Stack	

Instruction	Meaning	Before	After
ineg	$STACK[OPTOP] := -STACK[OPTOP];$ $PC := PC + 1$	(N)	(N)

Instruktionen der JVM

Ausdrücke

Stack Manipulation

Instruction	Meaning	Before	After
iconst_0	$OPTOP := OPTOP + 1;$ $STACK[OPTOP] := 0;$ $PC := PC + 1$	()	(N)
iconst_1	$OPTOP := OPTOP + 1;$ $STACK[OPTOP] := 1;$ $PC := PC + 1$	()	(N)
bipush <i>b</i>	$OPTOP := OPTOP + 1;$ $STACK[OPTOP] := b;$ $PC := PC + $ 2 	()	(N)
ldc <i>i</i>	$OPTOP := OPTOP + 1;$ $STACK[OPTOP] := \gamma_K^{CC}(i);$ $PC := PC + 2$	()	(N)
istore <i>v</i>	$STACK[VARs + v] := STACK[OPTOP];$ $OPTOP := OPTOP - 1;$ $PC := PC + 2$	(N)	()
iload <i>v</i>	$OPTOP := OPTOP + 1;$ $STACK[OPTOP] := STACK[VARs + v];$ $PC := PC + 2$	()	(N)

Instruktionen der JVM

Ausdrücke

Jumps

Instruction	Meaning	Before	After
goto a	$PC := a$	()	()
ifeq a	if ($STACK[OPTOP] = 0$) then $PC := a$ else $PC := PC + 3$ fi ; $OPTOP := OPTOP - 1$	(N)	()
if_icmpeq $a_1 a_2$	if ($STACK[OPTOP - 1] = STACK[OPTOP]$) then $PC := a_1$ else $PC := a_2$ fi ; $OPTOP := OPTOP - 2$	(N,N) (N)	()

Other conditional jumps include **ifne** ($\neq 0$), **ifle** (≤ 0), **ifge** (≥ 0), **if_icmpne** ($\neq 0$), **if_icmple** (≤ 0) and **if_icmpge** (≥ 0).

Übersetzung von Ausdrücken

Function		Restriction
$code(e_1 == e_2) \rho$	$= code\ e_1\ \rho; code\ e_2\ \rho; \mathbf{isub};$ $\mathbf{ifeq}\ l_1; \mathbf{iconst_0}; \mathbf{goto}\ l_2; l_1 : \mathbf{iconst_1}; l_2 :$	$Type(e_1) = Type(e_2) = N$
$code(e_1 \neq e_2) \rho$	$= code\ e_1\ \rho; code\ e_2\ \rho; \mathbf{isub};$ $\mathbf{ifeq}\ l_1; \mathbf{iconst_1}; \mathbf{goto}\ l_2; l_1 : \mathbf{iconst_0}; l_2 :$	$Type(e_1) = Type(e_2) = N$
$code(e_1 < e_2) \rho$	$= code\ e_1\ \rho; code\ e_2\ \rho; \mathbf{isub};$ $\mathbf{iflt}\ l_1; \mathbf{iconst_0}; \mathbf{goto}\ l_2; l_1 : \mathbf{iconst_1}; l_2 :$	$Type(e_1) = Type(e_2) = N$
$\vdots$		
$code(e_1 + e_2) \rho$	$= code\ e_1\ \rho; code\ e_2\ \rho; \mathbf{iadd}$	$Type(e_1) = Type(e_2) = N$
$code(e_1 - e_2) \rho$	$= code\ e_1\ \rho; code\ e_2\ \rho; \mathbf{isub}$	$Type(e_1) = Type(e_2) = N$
$code(e_1 * e_2) \rho$	$= code\ e_1\ \rho; code\ e_2\ \rho; \mathbf{imul}$	$Type(e_1) = Type(e_2) = N$
$code(e_1 / e_2) \rho$	$= code\ e_1\ \rho; code\ e_2\ \rho; \mathbf{idiv}$	$Type(e_1) = Type(e_2) = N$
$code(-e) \rho$	$= code\ e\ \rho; \mathbf{ineg}$	$Type(e) = N$
$code\ x\ \rho$	$= \mathbf{iload}\ \rho(x)$	x local variable of type N
$code\ c\ \rho$	$= \mathbf{ldc}\ \gamma_K^{-1}(c)$	c constant of type N
$code(x = e) \rho$	$= code\ e\ \rho; \mathbf{istore}\ \rho(x)$	$Type(e) = N, x$ local variable

Übersetzung von Kontrollflussanweisungen

$code(\text{if } e \text{ then } st_1 \text{ else } st_2 \text{ fi}) \rho$	$= code\ e\ \rho; \text{ ifeq } l_1; code\ st_1\ \rho; \text{ goto } l_2; l_1: code\ st_2\ \rho; l_2:$
$code(\text{if } e \text{ then } st \text{ fi}) \rho$	$= code\ e\ \rho; \text{ ifeq } l; code\ st\ \rho; l:$
$code\ (\text{while } e \text{ do } st \text{ od}) \rho$	$= l_1: code\ e\ \rho; \text{ ifeq } l_2; code\ st\ \rho; \text{ goto } l_1; l_2:$
$code\ (\text{do } st \text{ while } e) \rho$	$= l: code\ st\ \rho; code\ e\ \rho; \text{ ifne } l$
$code\ (st_1; st_2) \rho$	$= code\ st_1\ \rho; code\ st_2\ \rho$

$code\ (\text{for}(\text{init}; \text{condition}; \text{step}) \text{ body}) \rho =$	$code\ \text{init}\ \rho;$ $l_1: code\ \text{condition}\ \rho;$ $\text{ifeq } l_2;$ $code\ \text{body}\ \rho;$ $code\ \text{step}\ \rho;$ $\text{goto } l_1;$ $l_2:$
--	--

$\text{for}(\text{init}; \text{condition}; \text{step}) \text{ body}$

$\rightarrow \text{init}; \text{while } \text{condition} \text{ do } \text{body}; \text{step od}$

Übersetzung von SWITCH

Instruction	Meaning	Before	After
lookupswitch $d \ n$ $i_1 \ a_1$ $\dots$ $i_n \ a_n$	if ($STACK[OPTOP] = i_j$) (for smallest $1 \leq j \leq n$) then $PC := a_j$; else $PC := d$; fi; $OPTOP := OPTOP - 1$	(N)	()

Fall-Sprungziel
Paare

break Anweisungen
können innerhalb
der Blöcke $st_1, \dots, st_k$
sowie st verwendet
werden

$$\begin{array}{l}
 \text{code} \left(\begin{array}{l} \text{switch } (e) \{ \\ \text{case } i_1 : st_1; \\ \vdots \\ \text{case } i_k : st_k; \\ \text{default} : st \\ \} \end{array} \right) \rho = \begin{array}{l} \text{lookupswitch } l_d \ k \ i_1 \ l_1 \dots i_k \ l_k ; \\ l_1 : \text{code } st_1 \rho'; \\ \vdots \\ l_k : \text{code } st_k \rho'; \\ l_d : \text{code } st \rho'; \\ b : \end{array}
 \end{array}$$

where $\rho' = \rho[\lambda \mapsto b]$.

Erweitern der Adressumgebung
um ein Sprungziel für *break*
Anweisungen

Übersetzung der
break Anweisung

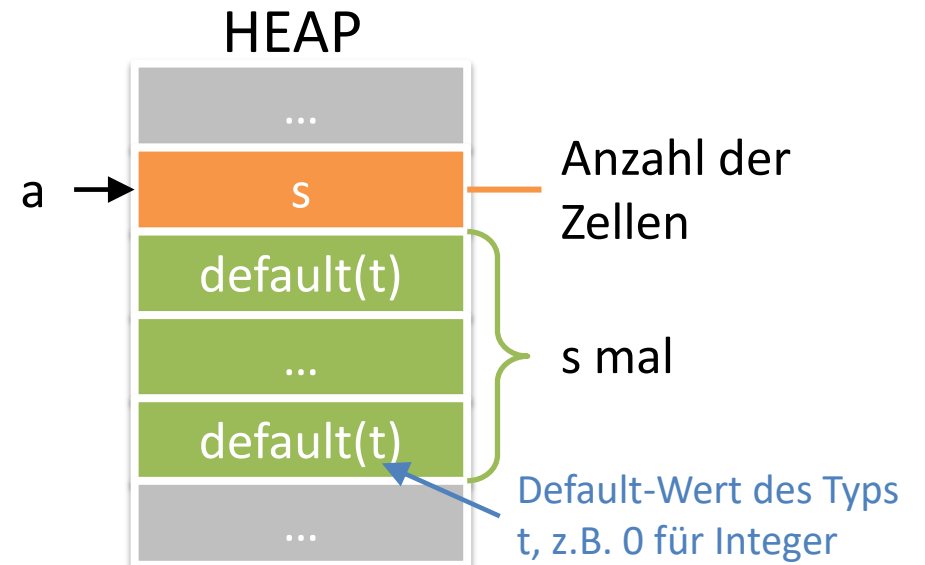
$\text{code break } \rho = \text{goto } \rho(\lambda)$

Eindimensionale Arrays

Instruction	Meaning	Before	After
newarray t	$STACK[OPTOP]$ $:= newArray(STACK[OPTOP], t);$ $PC := PC + 2$	(N) $t \in \{int, bool\}$	(A)
anewarray i	let $t = \gamma_K^{CC}(i)$ in $STACK[OPTOP]$ $:= newArray(STACK[OPTOP], t);$ $PC := PC + 3$ end	(N) t must be a class	(A)

newarray(s, t) liefert die Adresse a eines initialisierten Bereichs von $1 + s * sizeof(t)$ vielen aufeinanderfolgenden Zellen im HEAP.

sizeof(t) liefert die Anzahl der 4 Byte Zellen, die benötigt werden, um einen Wert vom Typ t zu speichern, wenn t ein skalarer Typ (int, bool) ist, oder um einen Pointer (Adresse) auf ein Objekt oder Array vom Typ t zu speichern, falls t ein Referenztyp ist.



Übersetzung von eindimensionalen Arrays

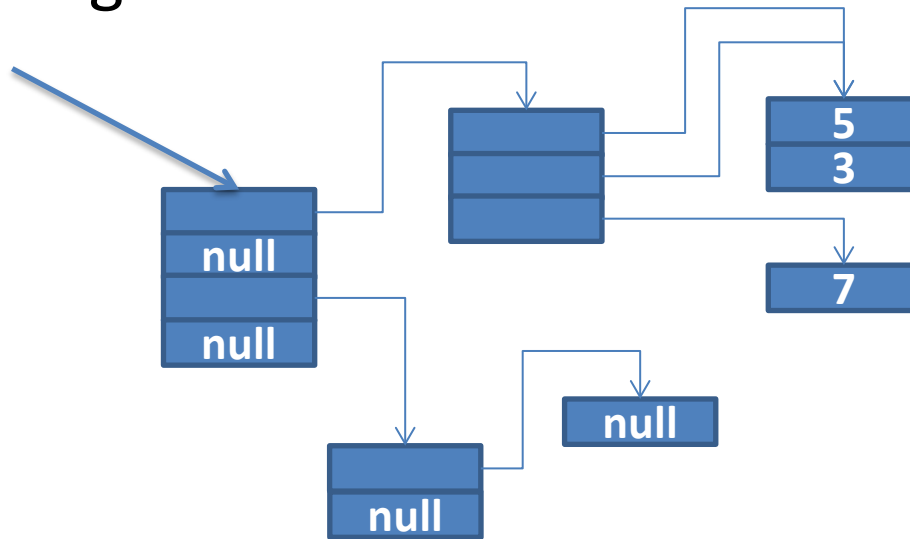
$code(\mathbf{new} \ t \ [e]) \ \rho = code \ e \ \rho; \mathbf{newarray} \ t$ where $t \in \{int, bool\}$

$code(\mathbf{new} \ c \ [e]) \ \rho = code \ e \ \rho; \mathbf{anewarray} \ i$ where c is a classname and $\gamma_K(i) = c$.

Instruction	Meaning	Before	After
$\mathbf{newarray} \ t$	$STACK[OPTOP]$ $\quad := newArray(STACK[OPTOP], t);$ $PC := PC + 2$	(N) $t \in \{int, bool\}$	(A)
$\mathbf{anewarray} \ i$	$\mathbf{let} \ t = \gamma_K^{CC}(i) \ \mathbf{in}$ $\quad STACK[OPTOP]$ $\quad \quad := newArray(STACK[OPTOP], t);$ $\quad PC := PC + 3$ $\mathbf{end}$	(N) t must be a class	(A)

Mehrdimensionale Arrays

- n-dimensionale Arrays sind implementiert als eindimensionale Arrays mit Verweisen auf (n-1)-dimensionale Arrays → zyklische Arrays nicht möglich
- Partielle mehrdimensionale nicht rechteckige Arrays möglich:



Mehrdimensionale Arrays

Anzahl zu initialisierender Dimensionen

Instruction	Meaning	Before	After
multianewarray i d	$\text{let } t = \gamma_K^{CC}(i)$ $(d_0, t_0) = \gamma_K^{CC}(t) \text{ in } +1$ $OPTOP := OPTOP - d;$ $STACK[OPTOP] := alloc(1, d, d_0, t_0);$ $PC := PC + 4$ end	$(N, \dots, N)$ d times	(A)

Eintrag des Arrays im Konstantenpool mit der Anzahl aller Dimension d_0 und dem Elementtyp t_0 .

Größe der Arrays für jede zu initialisierende Dimension

```

alloc(s, d, d0, t0) =
  if s = d0 then return newarray(STACK[OPTOP + s - 1], t0);
  r := newarray(STACK[OPTOP + s - 1], ArrayPointer);
  if s < d then
    for i := 0 to STACK[OPTOP + s - 1] - 1 do
      STORE[r+1+i] := alloc(s + 1, d, d0, t0);
  return r;

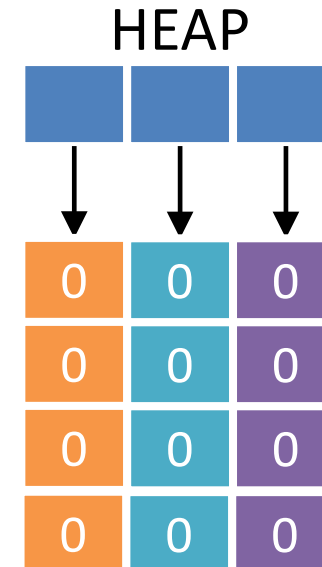
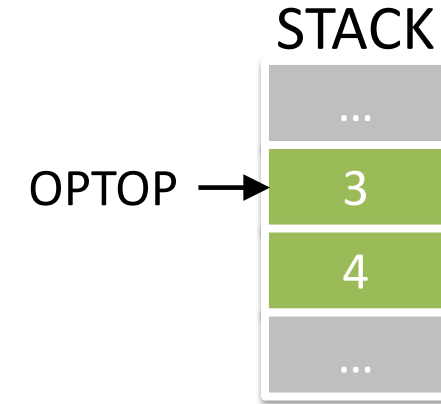
```

Verweis auf ein Array

Mehrdimensionale Arrays

Beispiel

- Quelltext: `int[][] matrix = new int[3][4];`
- Instruktion: `multianewarray i 2`
- Mit Hilfe des Konstantenpools γ_K^{CC} der aktuellen Klasse erhalten wir für i das Tupel (d_0, t_0)
- Weiterhin liegen die Größen beider Dimensionen des Arrays als Oberstes auf dem Keller. Nach zurücksetzen zeigt OPTOP auf den ersten Wert.
- Ablauf:
 - Erzeugen eines Arrays der Größe 3, welches je Zelle einen ArrayPointer beinhaltet.
 - Da noch nicht alle zu initialisierenden Dimensionen abgearbeitet sind ($s < d$) rufen wir `alloc(s + 1, d, d_0, t_0)` für alle Zellen des gerade erzeugten Arrays auf.
 - In den rekursiven Aufrufen ist nun $s == d_0$, d.h. die Arrays auf zweiter Ebene werden erzeugt und initialisiert.



Übersetzung von mehrdimensionalen Arrays

$code(\mathbf{new} \ t \ [e_1] \dots [e_k] \ \underbrace{[] \dots []}_{m\text{-times}}) \ \rho = code \ e_1 \ \rho; \dots code \ e_k \ \rho; \mathbf{multianewarray} \ i \ k$

where $\gamma_K(i) = (k + m, t)$.

Lesen von Arrays

Instruction	Meaning	Before	After
aload <i>v</i>	Lädt die Referenz eines Objekts bzw. Arrays $OPTOP := OPTOP + 1;$ $STACK[OPTOP] := STACK[VARs + v];$ $PC := PC + 2$	()	(O) or (A)
aaload	Lädt die Referenz in der i-ten Zelle eines Arrays $OPTOP := OPTOP - 1;$ if ($STACK[OPTOP] = NULL$) or ($STACK[OPTOP + 1] < 0$) or ($STACK[OPTOP + 1] > STORE[STACK[OPTOP]]$) then error(); fi; $STACK[OPTOP] :=$ $STORE[STACK[OPTOP] + STACK[OPTOP + 1] + 1];$ $PC := PC + 1$	(A,N)	(O) or (A)
iaload	Lädt die Zahl (integer) in der i-ten Zelle eines Arrays same as aaload	(A,N)	(N)

Referenz und Index i müssen auf dem Stack liegen

Übersetzen von Arrayzugriffen

$code\ x\ \rho = \mathbf{aload}\ \rho(x)$ falls x Variable von einem Referenztyp

$code\ x[i_1] \dots [i_k]\ \rho =$

```

aload  $\rho(x)$ ;
 $code\ i_1\ \rho$ ;
aaload;
 $\vdots$ 
 $code\ i_{k-1}\ \rho$ ;
aaload;
 $code\ i_k\ \rho$ ;
iaload;
  
```

Gilt nur für mehrdimensionale Arrays mit numerischem Inhalt. Wird ein Referenztyp im Array gespeichert, erhält man dessen Referenz mit `aaload` statt `iaload`.

Laden der Referenz auf das Array
 Laden aller benötigten verschachtelten Arrays
 Laden des numerischen Zellinhalts

Schreiben von Arrays

Instruction	Meaning	Before	After
astore <i>v</i>	$STACK[VARs + v] := STACK[OPTOP];$ $OPTOP := OPTOP - 1;$ $PC := PC + 2$	(O) or (A)	()
aastore	$OPTOP := OPTOP - 3;$ let $i = STACK[OPTOP + 2]$ in if ($STACK[OPTOP + 1] = NULL$) or ($i < 0$) or ($i > STORE[STACK[OPTOP + 1]]$) then $error();$ fi ; $STORE[STACK[OPTOP + 1] + i + 1] :=$ $STACK[OPTOP + 3];$ end $PC := PC + 1$	(A,N,O) or (A,N,A)	()
iastore	same as aastore	(A,N,N)	()

Schreibt die Referenz
in eine lokale Variable

Speichert die
Referenz auf
dem Stack in
der i-ten Zelle
eines Arrays

Referenz, Index i und
Referenz zum neuen
Inhalt müssen auf dem
Stack liegen